PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

ANTI-MONEY LAUNDERING SANCTIONS SCREENING SYSTEM

AN INTERNSHIP REPORT

Submitted by

JAHNU TANAI KUMAR HINDUPUR – 20221CAI0012

Under the guidance of,

Ms. SUMA N. G.

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Anti-Money Laundering Sanctions Screening System"** is a bonafide work of **JAHNU TANAI KUMAR HINDUPUR (20221CAI0012),** who has successfully carried out the internship work and submitted the report for partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (SPECIALIZATION DIVISION), ARTIFICIAL INTELLIGENCE & MACHINE LEARNING during 2025-2026.

20/4/2026

Ms. Suma N. G.
Assistant Professor
Internship Supervisor
PSCS (Spl. Division)
Presidency University

20/04/2026

Mr. Anandan Bellie
Assistant Professor
Program Internship Coordinator
PSCS (Spl. Division)
Presidency University

20/4/26.

Dr. Geetha A
Associate Professor
School Internship Coordinator
PSCS (Spl. Division)
Presidency University

20/04/2026

Dr. Zafar Ali Khan N
Professor & Head of the Department
PSCS (Spl. Division)
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS (Spl. Division)
Presidency University

Sl. No	Name of the Examiner	Signature
1.	Dr. Srinivasan TR	21/4/26
2.	Prof. IRSHAD KHAN	21/4/26

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING), at Presidency University, Bengaluru, named **Jahnu Tanai Kumar Hindupur**, hereby declare that the internship work titled **"Anti-Money Laundering Sanctions Screening System"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING (Spl. Division) (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Jahnu Tanai Kumar Hindupur USN: 20221CAI0012

PLACE: BENGALURU

DATE: 20 April 2026

INTERNSHIP COMPLETION CERTIFICATE

INNOVITEGRA SOLUTIONS

CIN: U62013KA2023PTC177507
GSTIN: 29AAHCI2695D1ZC
PAN: AAHCI2695D

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that Mr. **Jahnu Tanai Kumar Hindupur** has successfully completed an internship at our organization as a **Python and Android Developer Intern** from **5th January 2026 to 7th April 2026**.

During this period, he was involved in a project titled **Anti-Money Laundering Sanctions Screening System**, which was developed using Python, incorporating Artificial Intelligence and Machine Learning techniques. He worked on tasks such as data preprocessing, model training and evaluation, and the application of both supervised and unsupervised learning methods.

He exhibited a strong understanding of AI/ML fundamentals and consistently demonstrated initiative, problem-solving skills, and a collaborative attitude.

We appreciate his commitment and the quality of work delivered during the internship. We wish him continued success in his academic and professional journey.

Date of Issue: 7th April 2026
Place: Bangalore

Innovitegra Solutions Private Limited Bangalore

Authorized Signatory
Director

+91 87928 34949 www.innovitegra.com info@innovitegra.com A302, Sai Spring, Avalahalli, Yelahanka, Bangalore - 560064

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal Supervisor **Ms. Suma N. G.**, Assistant Professor, Presidency School of Computer Science and Engineering (Spl. Division), Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Zafar Ali Khan N,** Professor, Head of the Department, Presidency School of Computer Science and Engineering (Spl. Division)**,** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering (Spl. Division), and also the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Geetha A,** PSCS (Spl. Division), School Internship Coordinator and **Mr. Anandan Bellie,** Program Internship Coordinator**,** Presidency School of Computer Science and Engineering (Spl. Division), for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Jahnu Tanai Kumar Hindupur

ABSTRACT

The rapid expansion of digital financial systems has significantly increased the complexity of monitoring and preventing financial crimes such as money laundering and terrorist financing. Financial institutions are required to comply with strict international regulations and sanctions issued by global authorities such as the Financial Action Task Force (FATF), United Nations (UN), European Union (EU), and Office of Foreign Assets Control (OFAC). Traditional manual screening methods and legacy systems are no longer effective due to the large volume of data, diverse data formats, and the need for real-time decision-making. This has created a strong need for automated, scalable, and reliable Anti-Money Laundering (AML) systems that can efficiently identify high-risk entities while ensuring regulatory compliance.

This internship focuses on the design and development of an automated AML sanctions screening system that provides an end-to-end backend solution for financial compliance. The system adopts a pipeline-driven architecture to ingest and process sanctions data from multiple global sources available in formats such as XML, JSON, and CSV. A unified data normalization strategy is used to convert heterogeneous data into a consistent schema, which is then indexed using Elasticsearch for high-performance search. A hybrid matching approach is implemented, where candidate records are retrieved using search techniques and further refined using a Python-based scoring mechanism that evaluates name similarity along with contextual attributes such as date of birth and country. The system also includes a RESTful API layer for real-time screening, configurable risk categorization, and an ongoing monitoring mechanism for periodic re-evaluation.

The developed system demonstrates the ability to efficiently handle large-scale sanctions data and provide accurate and low-latency screening results. The hybrid scoring mechanism improves detection accuracy while reducing false positives, and the modular pipeline design ensures scalability and maintainability. Additional features such as audit logging, scheduling, and continuous monitoring enhance system reliability and compliance readiness. Overall, the project presents a practical and scalable AML screening solution that aligns with modern financial compliance requirements and industry standards.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
1.	Introduction	1
	1.1 Background	1
	1.2 Statistics of Internship	3
	1.3 Prior existing technologies	4
	1.4 Proposed approach	5
	1.5 Objectives	5
	1.6 SDGs	6
	1.7 Overview of internship report	
2.	Literature Review	8
3.	Methodology	12
4.	Project Management	18
	4.1 Internship timeline	18
	4.2 Risk analysis	19
	4.3 Internship budget	19
5.	Analysis and Design	22
	5.1 Requirements	22
	5.2 System Architecture	24
	5.3 System Flow Chart	26
	5.4 Choosing devices	27
	5.5 Designing units	27
	5.6 Standards	28
	5.7 Mapping with IoTWF reference model layers	28
	5.8 Domain model specification	29
	5.9 Communication model	29
	5.10 IoT deployment level	30

5.11 Functional view 30
5.12 Mapping IoT deployment level with functional view 30
5.13 Operational view 31
5.14 Other Design 31
6. Hardware, Software and Simulation 32
6.1 Hardware 32
6.2 Software development tools 32
6.3 Software code 33
6.4 Simulation 34
7. Evaluation and Results 36
7.1 Test points 36
7.2 Test plan 37
7.3 Test result 37
7.4 Insights 38
8. Social, Legal, Ethical, Sustainability and Safety Aspects 39
8.1 Social aspects 39
8.2 Legal aspects 40
8.3 Ethical aspects 40
8.4 Sustainability aspects 41
8.5 Safety aspects 41
9. Conclusion 43
References 45
Appendix 46

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Figure 1.1	AML Sanctions Screening System Architecture	2
2	Figure 1.2	AML Screening Processing Workflow	4
3	Figure 1.3	Sustainable Development Goals	6
4	Figure 3.1	Development Lifecycle	12
5	Figure 3.2	AML Processing Workflow	13
6	Figure 3.3	Layered Backend Architecture	13
7	Figure 3.4	Data Normalization Model	14
8	Figure 3.5	Testing Workflow	17
9	Figure 5.1	AML System Architecture	25
10	Figure 5.2	AML Processing Flow	26
11	Figure B.1	AML Individual Name Screening Page	50
12	Figure B.2	AML Business Name Screening Page	50
13	Figure B.3	AML Name Screening Results Page	51
14	Figure B.4	AML Risk Categories Range and Action Setup	51

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 1.1	Internship Summary	3
2	Table 2.1	Comparison of Existing Studies	10
3	Table 3.1	Supported Data Sources	16
4	Table 4.1	Internship Phase-wise Activities	18
5	Table 4.2	Risk Analysis	19
6	Table 4.3	Hardware Configuration	20
7	Table 5.1	Requirement Summary	24
8	Table 6.1	Software Tools Used	33
9	Table 6.2	Simulation Scenarios	34
10	Table 7.1	Test Cases	37
11	Table 7.2	Result Summary	38

ABBREVIATIONS

Abbreviation	Full Form
AML	Anti-Money Laundering
API	Application Programming Interface
BIS	Bureau of Industry and Security
CSV	Comma-Separated Values
DB	Database
DOB	Date of Birth
EU	European Union
FATF	Financial Action Task Force
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JSONL	JSON Lines
JWT	JSON Web Token
OFAC	Office of Foreign Assets Control
PEP	Politically Exposed Person
REST	Representational State Transfer
SQL	Structured Query Language
UN	United Nations
XML	Extensible Markup Language

Chapter 1
INTRODUCTION

1.1 Background

In modern financial environments, organizations handle a vast number of digital transactions across multiple platforms such as banking systems, fintech applications, and payment gateways. These systems generate large volumes of data and require strict regulatory compliance to ensure transparency and prevent financial crimes. One of the major challenges faced by such systems is detecting and preventing illicit activities such as money laundering and unauthorized financial transactions.

During my internship, I worked on understanding and implementing a backend system for Anti-Money Laundering (AML) sanctions screening that can efficiently process and analyse data from multiple sources. In financial compliance domains, sanctions data is obtained from various global authorities such as OFAC, United Nations (UN), European Union (EU), and other regulatory bodies. These datasets are available in different formats such as XML, JSON, CSV, and Excel, each having its own structure and complexity. The lack of a unified processing mechanism makes it difficult to integrate and screen data efficiently.

Traditional approaches such as manual screening or basic rule-based systems lead to inefficient workflows, where each dataset must be handled separately. This increases complexity and reduces scalability. To overcome these limitations, modern AML systems adopt automated backend architectures that use centralized pipelines for data ingestion, normalization, indexing, and screening which can be seen in Figure 1.1. During my internship, I focused on developing a backend AML sanctions screening system that demonstrates how such automated pipelines can efficiently process and analyse data from multiple watchlist sources.

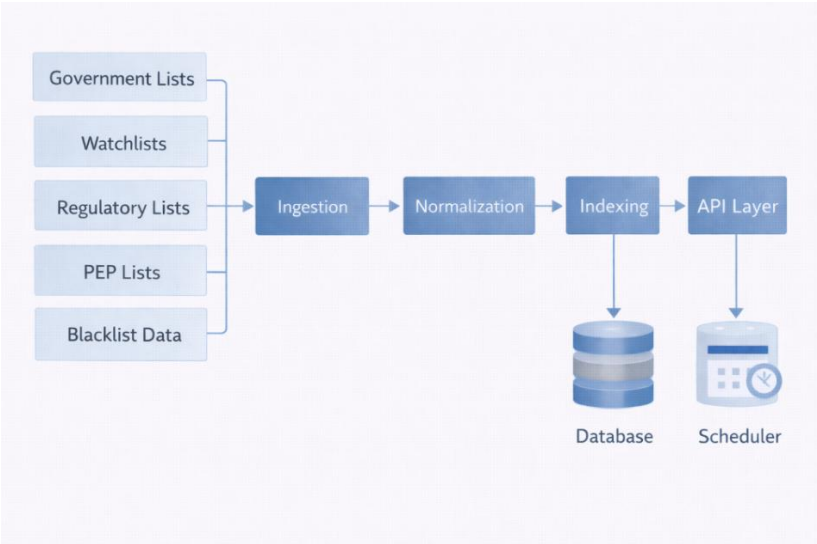

Figure 1.1: AML Sanctions Screening System Architecture

1.2 Statistics of Internship

The internship provided practical exposure to backend system development and financial compliance solutions. Over a duration of approximately 12 weeks, I worked as a Python Developer Intern at Innovitegra Solutions Pvt. Ltd., Bangalore, designing, implementing, and testing an AML sanctions screening system capable of handling multiple data sources and providing real-time screening functionality.

In recent studies on Anti-Money Laundering systems, it has been observed that the increasing volume of financial transactions and the dynamic nature of sanctions data require automated and scalable screening solutions. Research highlights that traditional rule-based systems often fail to handle variations in entity names and large datasets efficiently. During the internship, I observed similar challenges while working with heterogeneous sanctions data formats such as XML, JSON, and CSV, which required a structured normalization and processing approach.

Furthermore, existing literature emphasizes the importance of combining high-performance search engines with intelligent matching techniques to improve accuracy and reduce false positives. This observation aligns with the approach implemented during the internship, where Elasticsearch was used for efficient candidate retrieval, followed by a scoring mechanism that incorporates name similarity and contextual attributes. The results obtained during development confirmed that such hybrid approaches are effective in real-world AML screening scenarios.

The key outcomes of the internship are as follows:

- Development of a backend AML sanctions screening system using Python and Flask
- Integration of multiple sanctions sources including OFAC, UN, EU, BIS, and OpenSanctions, along with support of custom list uploads by the user.
- Implementation of a data normalization pipeline for handling XML, JSON, CSV, and Excel formats
- Development of RESTful APIs for screening, configuration, and monitoring
- Design of database schemas for risk categories, customer monitoring, and source tracking
- Implementation of search and matching logic using Elasticsearch
- Execution of multiple test scenarios to validate system performance and accuracy

The major activities and outcomes of the internship are summarized in Table 1.1.

Table 1.1: Internship Summary

Parameter	Details
Duration	12 Weeks
Technology	Python, Flask, PostgreSQL, Elasticsearch
Sources Supported	OFAC, UN, EU, BIS, OpenSanctions, Custom Lists
APIs Developed	20+ REST APIs
Core Modules	Data Ingestion, Normalization, Screening, Risk Configuration, Monitoring
Testing	Multiple screening and validation test cases

1.3 Prior Existing Technologies

Before the adoption of modern AML screening systems, financial institutions relied on traditional approaches that presented several limitations.

1. **Manual Screening:** In this approach, customer names are manually checked against sanctions lists. While simple for small-scale use, it becomes highly inefficient as data volume increases. The process is time-consuming and prone to human errors.

2. **Rule-Based Systems:** These systems rely on exact matching techniques to identify sanctioned entities. Although faster than manual methods, they fail to detect variations in names, aliases, or spelling differences, leading to missed matches or inaccurate results.
3. **Legacy AML Systems:** Earlier systems used static databases that were not updated frequently. This resulted in outdated screening data and increased compliance risks, as new sanctions were not reflected in real time.
4. **Limitations:**
   - High processing time and inefficiency
   - Inability to handle large-scale data
   - High false-positive and false-negative rates
   - Lack of real-time updates
   - Poor scalability and flexibility

These limitations highlight the need for an automated system that can efficiently process multiple data sources and provide accurate screening results.

1.4 Proposed Approach

During the internship, I proposed and implemented a backend-driven AML sanctions screening system that uses a pipeline-based architecture to automate data processing and screening.

The system workflow is illustrated in Figure 1.2, which shows how data flows through different stages of processing.

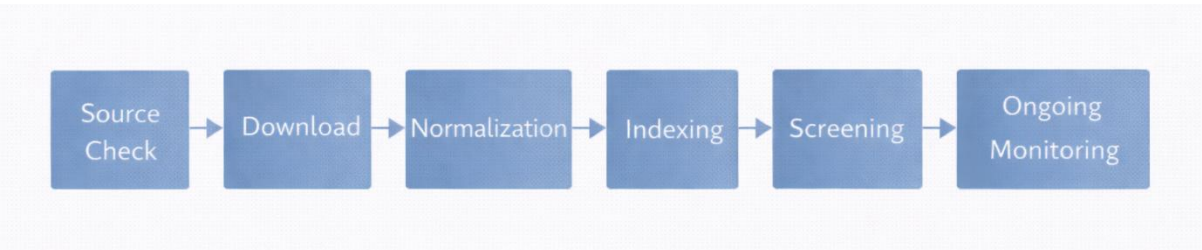

Figure 1.2: AML Screening Processing Workflow

The system follows a structured processing pipeline:

1. Request Handling: The system receives screening requests through RESTful API endpoints.
2. Data Acquisition and Validation: Sanctions data is collected from multiple sources and validated for consistency and completeness.
3. Data Normalization: All input data is converted into a unified format to ensure consistency across different sources.
4. Indexing and Search: Normalized data is indexed into Elasticsearch for efficient retrieval.
5. Screening Logic: The system performs matching using search-based retrieval and scoring techniques.
6. Response Generation: The results are processed and returned as structured responses with risk categorization.

This approach reduces complexity by ensuring that all data processing follows a centralized pipeline instead of multiple independent processes.

1.5 Objectives

The main objectives of the internship work are:

- To design and implement an AML sanctions screening system
- To integrate multiple global sanctions data sources
- To develop a unified data normalization mechanism
- To implement efficient search and matching techniques
- To develop RESTful APIs for system interaction
- To support real-time and ongoing screening processes
- To ensure scalability, modularity, and maintainability

1.6 Sustainable Development Goals (SDGs)

The internship work aligns with several Sustainable Development Goals as shown in Fig 1.3 by promoting secure and transparent financial systems.

- SDG 9: Industry, Innovation, and Infrastructure - The development of an AML screening system contributes to building scalable and reliable digital infrastructure for financial compliance.
- SDG 8: Decent Work and Economic Growth - Efficient financial systems improve operational performance and support economic stability.
- SDG 16: Peace, Justice and Strong Institutions - The system helps prevent illicit financial activities and strengthens regulatory compliance.

Figure 1.3: Sustainable Development Goals

1.7 Overview of Internship Report

This report is organized into multiple chapters to provide a structured and comprehensive understanding of the internship work, system design, and implementation of the AML sanctions screening system.

Chapter 1 - Introduction: This chapter presents the background of financial compliance challenges, the need for AML screening systems, and an overview of the developed solution. It also outlines the objectives, scope, and structure of the report.

Chapter 2 - Literature Review: This chapter reviews existing AML screening approaches, data processing techniques, and system architectures. It identifies limitations in traditional systems and highlights the need for automated solutions.

Chapter 3 - Methodology: This chapter explains the development methodology adopted during the internship. It describes the design approach, system workflow, and implementation strategy of the AML screening pipeline.

Chapter 4 - Project Management: This chapter focuses on the planning and management aspects of the internship. It includes the timeline, task organization, and risk analysis.

Chapter 5 - Analysis and Design: This chapter provides a detailed analysis of system requirements and presents the architecture of the AML screening system, including diagrams and models.

Chapter 6 - Hardware, Software and Simulation: This chapter describes the tools, technologies, and environment used for development and testing.

Chapter 7 - Evaluation and Results: This chapter presents the testing process, results, and performance evaluation of the system.

Chapter 8 - Social, Legal, Ethical, Sustainability and Safety Aspects: This chapter discusses the broader implications of the system in financial, legal, and ethical contexts.

Chapter 9 - Conclusion: This chapter summarizes the outcomes of the project and suggests possible future enhancements.

Chapter 2
LITERATURE REVIEW

The field of Anti-Money Laundering (AML) has evolved significantly with the rapid expansion of digital financial systems and the increasing complexity of global regulatory requirements. During my internship, I studied various research works, regulatory frameworks, and technical approaches related to sanctions screening, search-based matching systems, and financial compliance infrastructures. This chapter presents a comprehensive review of relevant literature, followed by identification of key gaps that motivated the development of the AML sanctions screening system.

The Financial Action Task Force (FATF) guidelines [1] emphasize the importance of risk-based approaches and continuous monitoring of customers against global sanctions lists such as OFAC, United Nations (UN), and European Union (EU). These frameworks highlight the necessity of automated systems capable of handling large-scale financial data. However, they primarily define compliance requirements and lack implementation-level guidance for building scalable AML systems.

OpenSanctions datasets and global watchlist aggregators [2] provide comprehensive and structured data for regulatory, politically exposed persons (PEPs), and crime-related entities. These datasets simplify access to sanctions information but do not define how such data should be normalized and processed in real-time systems.

Elasticsearch-based search architectures [3] have been widely used for handling large-scale text retrieval problems. These systems use inverted indexing and tokenization to enable fast and efficient search operations. While they significantly improve performance, they lack contextual intelligence required for AML decision-making.

Levenshtein distance and fuzzy string-matching techniques [4] are commonly applied for approximate string matching in name-based screening systems. These methods improve detection of variations and spelling differences. However, when used independently, they often produce high false-positive results due to lack of contextual filtering.

Soundex and phonetic matching algorithms [5] have also been explored for identifying names with similar pronunciation. These techniques are useful for handling transliteration differences but may fail in multilingual datasets or complex name structures.

Relational database systems such as PostgreSQL [6] are widely used for storing structured data, including customer records, risk categories, and audit logs. While they provide reliability and consistency, they are not optimized for full-text search and approximate matching.

Data pipeline architectures [7] are commonly used in modern systems to automate ingestion, processing, and transformation of large datasets. These pipelines improve scalability and reduce manual effort. However, many implementations lack integration with real-time APIs and continuous monitoring systems.

RESTful API architectures [8] enable seamless communication between backend systems and client applications. They provide scalability and flexibility for system integration. However, many research implementations do not integrate APIs with complex backend processing pipelines.

Scheduling and monitoring systems [9] are essential for continuous compliance in AML applications. These systems enable periodic re-screening of customers to detect changes in risk status. However, many systems lack proper integration with screening engines and alert mechanisms.

Hybrid AML architectures [10] combine multiple technologies such as data pipelines, search engines, and scoring mechanisms to improve overall system performance. While these architectures are conceptually strong, many lack practical implementation details.

A comparative analysis of the reviewed approaches is presented in Table 2.1.

Table 2.1: Comparison of Existing Studies

Reference	Focus Area	Limitation	Solution in Internship
[1] FATF Guidelines	Regulatory Compliance	No system implementation	Built automated AML backend system
[2] OpenSanctions	Data Sources	No processing pipeline	Implemented normalization pipeline
[3] Elasticsearch	Search & Indexing	No contextual scoring	Added hybrid scoring mechanism
[4] Levenshtein	String Matching	High false positives	Combined with contextual attributes
[5] Soundex	Phonetic Matching	Limited accuracy	Used hybrid similarity approach
[6] PostgreSQL	Data Storage	Poor text search	Integrated with Elasticsearch
[7] Data Pipelines	Data Processing	No real-time integration	Built end-to-end pipeline
[8] REST APIs	System Communication	Limited backend integration	Developed API-driven system
[9] Scheduling Systems	Monitoring	No alert integration	Added ongoing monitoring module
[10] Hybrid AML Systems	System Design	Lack of practical implementation	Built complete integrated system

As shown in Table 2.1, most existing research focuses on individual components such as regulatory frameworks, search engines, or matching algorithms, but lacks a unified system that integrates all functionalities into a single platform.

From the reviewed literature, several critical gaps can be identified in existing AML systems. A major limitation is the absence of integrated systems capable of handling multiple sanctions data formats within a single architecture. While data sources and regulatory frameworks are

well established, there is limited guidance on combining them with real-time processing systems.

Another key gap is the lack of integration between search-based retrieval systems and contextual scoring mechanisms. Technologies such as Elasticsearch and fuzzy matching techniques improve efficiency and detection capability, but when used independently, they often result in high false positives due to the absence of filtering based on additional attributes such as date of birth, nationality, or risk category.

In addition, existing data pipeline architectures often focus on batch processing and do not adequately support real-time screening or seamless API integration. This creates challenges in handling continuously evolving financial data and performing instant compliance checks.

Furthermore, continuous monitoring and alert mechanisms are insufficiently implemented in many systems. Although scheduling tools exist, they are not always effectively integrated with screening engines to enable proactive detection of changes in risk status.

These limitations highlight the need for a unified AML system that integrates data ingestion, normalization, search, scoring, API interaction, and monitoring into a cohesive and scalable architecture.

During my internship, these identified gaps were addressed by developing a comprehensive AML sanctions screening system that combines automated data ingestion, normalization, indexing, and screening within a single pipeline-driven architecture. The system integrates search-based retrieval with contextual scoring, supports multiple data formats, and provides RESTful APIs for real-time interaction. Additionally, the inclusion of ongoing monitoring and alert mechanisms ensures continuous compliance and improved risk detection.

Overall, the developed system demonstrates how a unified, scalable, and modular backend architecture can effectively overcome the limitations of existing approaches, providing a practical and efficient solution for modern AML compliance requirements.

Chapter 3
METHODOLOGY

During the internship, I followed a structured and systematic approach to design and implement the Anti-Money Laundering (AML) Sanctions Screening System. The methodology focused on building a modular, scalable, and efficient backend system capable of handling multiple sanctions data sources such as OFAC, United Nations (UN), European Union (EU), BIS, and OpenSanctions. The development process emphasized pipeline-driven architecture, data normalization, hybrid screening logic, and continuous monitoring.

Development Approach

The development of the AML system was carried out using an iterative and modular approach, where each component was designed, implemented, and tested independently before being integrated into the complete system. Initially, requirement analysis was performed to understand AML compliance needs, including sanctions screening, data freshness, and ongoing monitoring. Based on this, a layered and pipeline-driven architecture was designed to separate data ingestion, normalization, indexing, screening, and monitoring processes.

The implementation phase involved developing backend APIs using Python and Flask, followed by integration of modules such as source checking, data download, normalization, indexing, and screening logic. The system was then tested using multiple screening scenarios to ensure accuracy, performance, and reliability. The overall development lifecycle is summarized in Figure 3.1, which shows the step-by-step process followed during the internship.

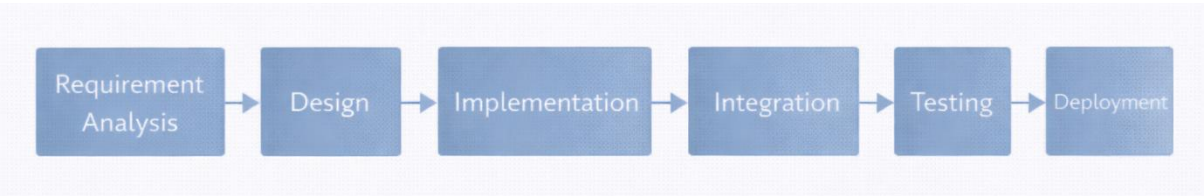

Figure 3.1: Development Lifecycle

System Workflow

The AML system follows a structured workflow for processing sanctions data and screening requests. This workflow ensures that each stage of processing is clearly defined and executed in a systematic manner. As illustrated in Figure 3.2, the system begins by checking external sources for updates. If updates are detected, the data is downloaded and processed through normalization. The normalized data is then indexed into Elasticsearch for efficient search operations. When a screening request is received through an API endpoint, the system retrieves candidate records, computes match scores, and returns results with risk categorization.

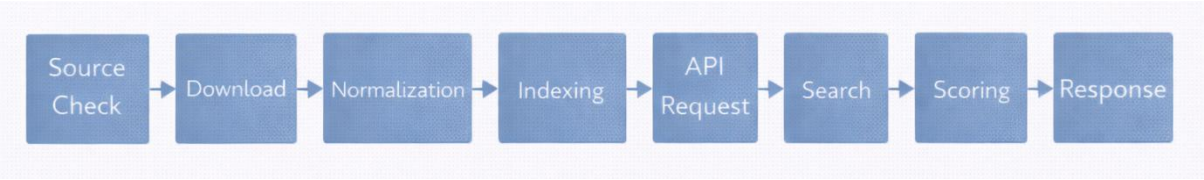

Figure 3.2: AML Processing Workflow

Backend Architecture Design

During the internship, I designed a layered backend architecture to ensure modularity, scalability, and maintainability. Each layer is responsible for a specific function in the system. The architecture is shown in Figure 3.3, illustrating the interaction between the different layers.

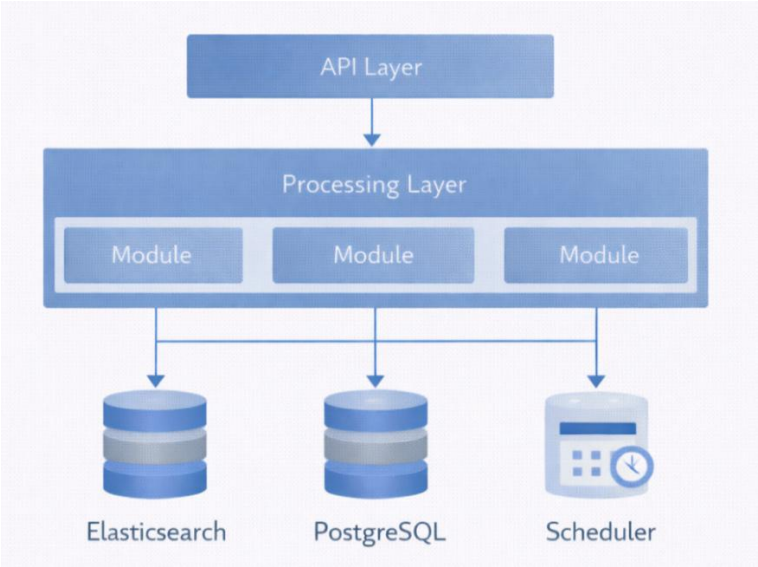

Figure 3.3: Layered Backend Architecture

The key layers include:

- API Layer: Handles incoming HTTP requests and exposes endpoints for screening and configuration
- Validation Layer: Ensures correctness of input data and request structure
- Processing Layer: Implements screening logic and scoring mechanisms
- Search Layer: Uses Elasticsearch for candidate retrieval and indexing
- Database Layer: Stores structured data such as risk categories, logs, and schedules
- Monitoring Layer: Handles ongoing screening and alert generation

This architecture ensures that each component operates independently, improving maintainability and scalability.

Data Normalization Strategy

A core aspect of the methodology is the use of a unified data normalization process, which acts as an intermediate representation for all sanctions data. Instead of directly processing different formats, the system follows a two-step transformation process:

Source Format → Normalized JSONL → Indexed Data

This process is illustrated in Figure 3.4.

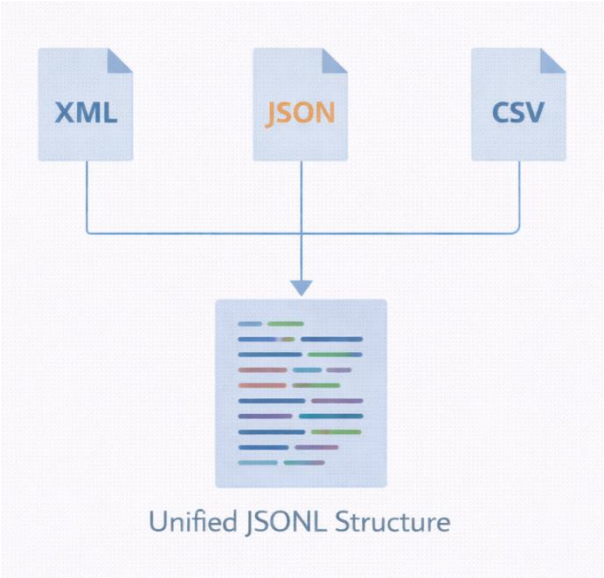

Figure 3.4: Data Normalization Model

The normalization process ensures:

- Consistent data representation across all sources
- Simplified processing and indexing
- Easy integration of new data sources
- Improved search efficiency

This approach is essential for handling heterogeneous sanctions datasets.

Hybrid Screening Mechanism

To achieve both speed and accuracy, the system implements a hybrid screening approach consisting of two stages:

- Candidate Retrieval: Elasticsearch retrieves potential matches based on name and alias search
- Scoring Mechanism: A weighted scoring algorithm evaluates similarity

The final score is calculated using:

- Name Similarity (70%)
- Date of Birth Match (20%)
- Country Match (10%)

This approach improves detection accuracy while reducing false positives.

Request Validation and Security

Validation and security are critical components of the AML system. During the internship, validation mechanisms were implemented to ensure that incoming requests meet required constraints.

The validation process checks:

- Presence of required fields
- Data format correctness
- Request structure

Security is implemented through controlled API access and request validation mechanisms to ensure that only authorized and valid requests are processed.

Data Handling from Multiple Sources

The system supports multiple external sanctions sources, which are summarized in Table 3.1.

Table 3.1: Supported Data Sources

Data Source	Description
OFAC	US sanctions list
United Nations	Global sanctions list
European Union	EU financial sanctions
BIS	Denied persons list
OpenSanctions	Aggregated global datasets

As shown in Table 3.1, the system is capable of handling multiple global sanctions sources, making it suitable for real-world AML compliance scenarios.

Error Handling Strategy

To ensure system reliability, robust error handling mechanisms were implemented. The system detects errors at different stages such as data ingestion, normalization, and screening.

Errors are handled by:

- Returning structured error responses
- Logging errors for debugging and traceability
- Preventing system crashes through controlled handling

This ensures stable system behaviour even under invalid or unexpected inputs.

Testing Methodology

The system was tested using a combination of manual and scenario-based testing approaches.

The testing process involved:

- Sending API requests using tools such as Postman
- Testing different screening scenarios
- Verifying correctness of match results
- Evaluating error handling and edge cases

The testing workflow is illustrated in Figure 3.5.

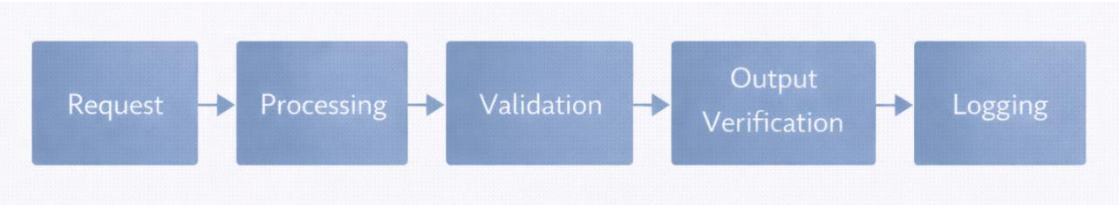

Figure 3.5: Testing Workflow

This approach ensured that the system performs accurately and reliably under different operational conditions.

Chapter 4
PROJECT MANAGEMENT

During the internship, effective project management played a crucial role in successfully designing and implementing the Anti-Money Laundering (AML) Sanctions Screening System. I followed a structured approach to plan, execute, and monitor the development process within the given timeframe. The project management activities included timeline planning, risk analysis, and efficient utilization of available resources.

4.1 Internship Timeline

The internship was carried out over a period of approximately 12 weeks, during which I systematically completed different phases of development. The work was divided into multiple stages to ensure proper planning and execution. The detailed breakdown of tasks performed during each phase is provided in Table 4.1.

Table 4.1: Internship Phase-wise Activities

Phase	Duration	Activities
Requirement Analysis	Week 1	Understanding AML concepts, sanctions screening requirements
System Design	Week 2-3	Designing system architecture and data pipeline
Development	Week 4-8	Implementing APIs, data ingestion, normalization, indexing
Integration & Testing	Week 9-10	Testing screening logic, APIs, and system workflows
Deployment & Optimization	Week 11-12	Deployment, performance tuning, and documentation

As shown in Table 4.1, I followed a structured development approach, ensuring that each phase was completed before moving to the next. This helped in managing time effectively and maintaining consistency in development.

4.2 Risk Analysis

During the internship, I identified several potential risks that could affect the development and performance of the AML system. Proper risk analysis helped minimize issues to ensure smooth project execution. The major risks and their mitigation strategies are summarized in Table 4.2.

Table 4.2: Risk Analysis

Risk Type	Description	Mitigation Strategy
Technical Risk	Handling multiple sanctions data formats	Used normalization pipeline and modular design
Integration Risk	Combining multiple system components	Performed incremental integration
Performance Risk	Delay in large-scale screening operations	Optimized search and indexing using Elasticsearch
Security Risk	Unauthorized API access	Implemented authentication and validation
Data Risk	Frequent updates in sanctions sources	Implemented automated update pipeline

The major technical challenge during the internship was handling multiple sanctions data sources and ensuring accurate screening results. To overcome this, I adopted a pipeline-based architecture combined with search and scoring mechanisms, which improved both efficiency and accuracy.

4.3 Internship Budget

The development of the AML Sanctions Screening System was carried out using a cost-effective approach, primarily relying on open-source tools and available resources. This ensured that the system could be developed without requiring significant financial investment.

Hardware Resources

The development and deployment of the AML sanctions screening system were carried out across multiple computing environments, each serving a specific purpose in the system lifecycle, including development, testing, and production deployment.

During the initial phase of the internship, system development was performed on a personal laptop. This environment was sufficient for implementing core functionalities such as data ingestion, normalization pipelines, API development, and integration with database and search systems. However, due to limitations in computational resources, it was not suitable for large-scale data processing or performance-intensive operations.

For testing and performance validation, the system was deployed on the organization's high-performance GPU server running Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-14-generic x86_64). This environment provided significantly higher computational power, enabling efficient handling of large datasets, faster indexing in Elasticsearch, and improved execution of screening processes.

Finally, the system was deployed on the client server environment, which has a configuration similar to the testing server. This ensured consistency between testing and production environments, allowing reliable performance, scalability, and continuous monitoring in real-world usage scenarios. The overall hardware configurations used across different stages are summarized in Table 4.3.

Table 4.3: Hardware Configuration

Component	Development System	Testing Server	Client Server
CPU	Intel i9-13980HX	Intel Xeon Silver 4410Y (multi-core)	Intel i9-13900H
RAM	16 GB	75 GB	64 GB
Storage	1 TB SSD	200 GB (virtual disk)	2 TB NVMe SSD
GPU	8 GB Dedicated GPU	Not primary / virtualized	NVIDIA RTX 4080
OS	Windows 11	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS

Software Resources

The system was developed using open-source software tools:

- Python programming language
- Flask framework
- PostgreSQL database
- Elasticsearch search engine
- Postman for API testing

Since all tools used were open-source, the overall cost of development was minimal.

Chapter 5
ANALYSIS AND DESIGN

5.1 Requirements

During the internship, I performed a comprehensive requirement analysis to design an Anti-Money Laundering (AML) Sanctions Screening System capable of handling real-world financial compliance scenarios. The system was required to process multiple sanctions data sources and provide a unified screening pipeline. The requirements were categorized into functional requirements and non-functional requirements to ensure both system capability and performance.

5.1.1 Functional Requirements

The primary functional requirement of the system is to support screening against multiple global sanctions sources such as OFAC, United Nations (UN), European Union (EU), BIS, and OpenSanctions. Each source provides data in different formats such as XML, JSON, and CSV, making it necessary for the system to handle heterogeneous data efficiently.

Another key requirement is real-time screening capability. The system must provide an API endpoint that allows instant screening of customer names against the latest sanctions data. This ensures compliance during onboarding and transaction processing.

A critical requirement identified during the internship is the implementation of a data normalization process. Instead of directly processing different formats, the system must transform all incoming data into a unified JSONL structure. This standardized format simplifies indexing and search operations.

The system must also support automated data ingestion. It should continuously monitor sanctions sources, detect updates, and download new data when required. This ensures that screening is always performed on the latest available data.

Another essential requirement is the implementation of a hybrid screening mechanism. The system must retrieve candidate matches using a search engine and then apply a scoring algorithm based on name similarity, date of birth, and country.

The system must also expose RESTful API endpoints for interaction. These APIs allow external systems to perform screening, retrieve watchlists, configure risk categories, and manage ongoing monitoring.

Finally, the system must support ongoing monitoring of customers. It should periodically re-screen saved entities and generate alerts when risk levels change.

5.1.2 Non-Functional Requirements

In addition to functional capabilities, several non-functional requirements were identified to ensure system performance and reliability.

One of the most important non-functional requirements is performance. The system must provide fast response times for screening requests. This is achieved by using Elasticsearch for high-speed search operations.

Another key requirement is scalability. The system should handle multiple concurrent requests efficiently and support increasing data volume from multiple sanctions sources.

Modularity is also essential. The system should be designed in a way that allows independent development of components such as ingestion, normalization, indexing, and screening.

Security is critical in AML systems. The system must ensure controlled access to APIs and protect sensitive data through validation and authentication mechanisms.

Reliability is another important requirement. The system must operate continuously, ensuring consistent results and handling failures gracefully.

Finally, maintainability is required to support future enhancements. The system should follow clean design principles with proper structure and documentation.

5.1.3 Summary of Requirements

The overall requirements of the system are summarized in Table 5.1.

Table 5.1: Requirement Summary

Category	Requirement	Description
Functional	Multi-source support	OFAC, UN, EU, BIS, OpenSanctions
	Real-time screening	API-based instant screening
	Data normalization	Unified JSONL representation
	Automated pipeline	Source monitoring and updates
	Hybrid screening	Search + scoring mechanism
	APIs	REST endpoints for interaction
	Monitoring	Ongoing customer screening
Non-Functional	Performance	Fast response using Elasticsearch
	Scalability	Handle large data and requests
	Security	Authentication and validation
	Maintainability	Modular system design

As shown in Table 5.1, the system requirements focus on both functionality and performance, ensuring that the AML system is capable of handling complex compliance scenarios efficiently.

5.2 System Architecture

During the internship, I designed a comprehensive backend architecture for the AML Sanctions Screening System to handle multi-source data processing in a structured and scalable manner. The architecture is based on a modular and layered design, ensuring clear separation of responsibilities and efficient processing of incoming requests.

The complete architecture of the system is illustrated in Figure 5.1.

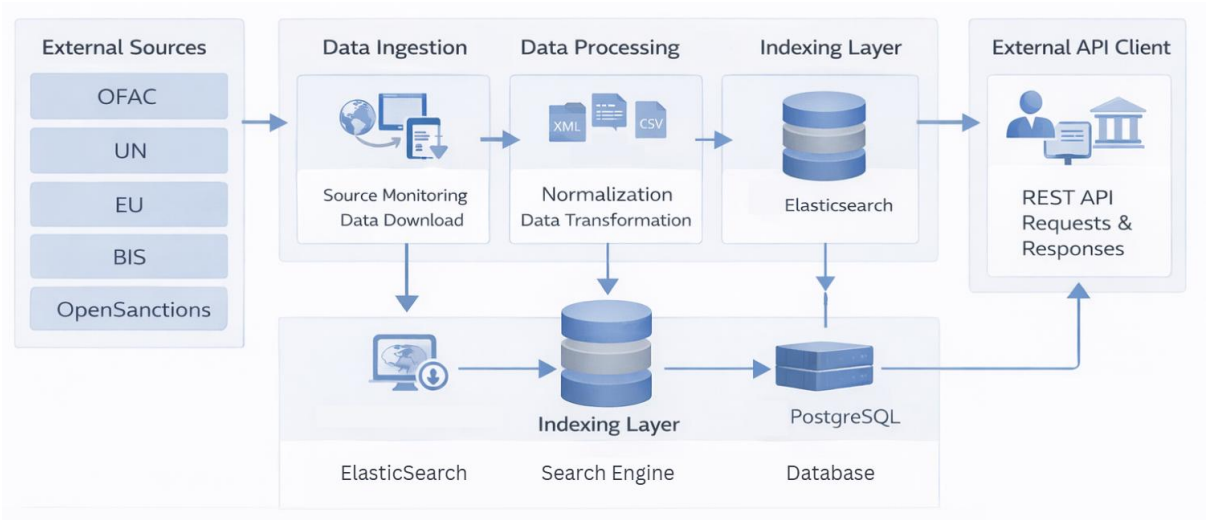

Figure 5.1: AML System Architecture

As shown in Figure 5.1, the system is organized into multiple layers and components, each responsible for a specific part of the processing pipeline.

1. External Sources Layer: This layer includes global sanctions providers such as OFAC, UN, EU, BIS, and OpenSanctions. These sources provide raw data in multiple formats.
2. Data Ingestion Layer: This layer monitors sources for updates and downloads new data when available. It ensures data freshness and reduces redundant processing.
3. Data Processing Layer: This layer performs normalization of heterogeneous data into a unified schema. It ensures consistency and prepares data for indexing.
4. Indexing Layer: The normalized data is indexed into Elasticsearch. Versioned indices and alias switching ensure zero-downtime updates.
5. Application Layer: This layer includes Flask APIs and screening logic. It handles real-time requests and computes match scores.
6. Database Layer: PostgreSQL is used to store metadata such as risk categories, logs, schedules, and alerts.
7. Monitoring Layer: This layer performs ongoing screening and generates alerts based on risk changes.

8. End-to-End Data Flow: The architecture supports a complete flow:
    a. Source monitoring
    b. Data download
    c. Normalization
    d. Indexing
    e. API screening
    f. Monitoring and alerts
9. Advantages of the Architecture
    a. Scalability: Handles large datasets
    b. Modularity: Independent components
    c. Reliability: Continuous operation
    d. Flexibility: Supports multiple sources
    e. Efficiency: Optimized search and processing

5.3 System Flow Chart

The AML system follows a structured workflow to process data and screening requests efficiently. The flow begins with checking sources for updates. If updates are found, data is downloaded, normalized, and indexed. When a screening request is received, the system retrieves candidates using Elasticsearch and applies scoring logic before returning results.

This flow can be observed in Fig 5.2 which shows how all data passes through a consistent pipeline, enabling efficient processing and accurate screening.

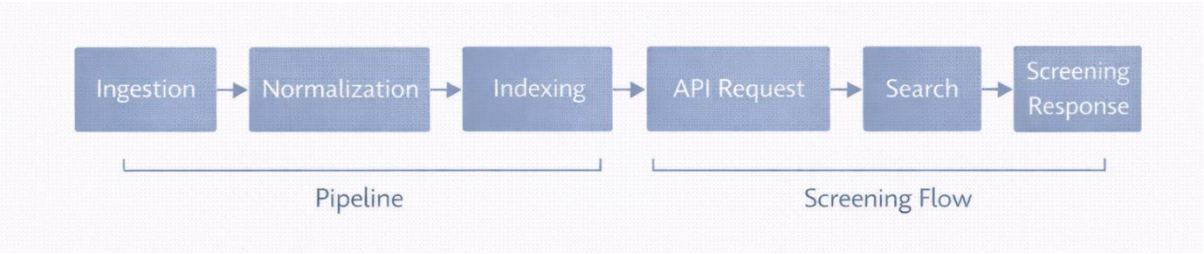

Figure 5.2: AML Processing Flow

5.4 Choosing Devices

During the internship, the system was developed and tested using standard hardware suitable for backend application development and data processing tasks. The primary development environment consisted of a personal laptop with moderate specifications, which was sufficient for implementing APIs, data processing pipelines, and testing system functionality.

- Development system: Personal laptop (Intel i9 or equivalent)
- Memory: 16 GB RAM
- Storage: SSD for faster data access

For deployment and real-world usage, the system was designed to run on a server-based environment. A Linux-based server (Ubuntu) was preferred due to its stability, compatibility with backend technologies, and support for continuous service execution.

For enterprise-level deployment, the system can be scaled using higher configurations such as multi-core processors, increased RAM (32-64 GB), and high-speed storage. These enhancements enable efficient handling of large datasets, faster indexing in Elasticsearch, and improved API performance under high load.

Thus, the chosen devices ensure that the system remains cost-effective during development while being scalable for production environments.

5.5 Designing Units

During the internship, the AML system was designed using a modular approach, where the backend is divided into multiple functional units. Each unit is responsible for a specific task in the overall processing pipeline, ensuring clear separation of concerns and efficient execution.

The main units include:

- **Data Acquisition Unit:** Responsible for monitoring external sanctions sources, detecting updates, and downloading data
- **Data Processing Unit:** Performs normalization of heterogeneous data formats into a unified structure
- **Index Management Unit:** Handles creation and updating of Elasticsearch indices

- **Screening Logic Unit:** Implements candidate retrieval and scoring mechanisms
- **API Unit:** Handles incoming requests and response generation
- **Monitoring Unit:** Performs ongoing screening and alert generation

Each unit operates independently but interacts with others through a well-defined workflow. This modular design improves scalability, maintainability, and flexibility, allowing new features or components to be added without affecting the entire system.

5.6 Standards

During the internship, the AML system was designed to follow widely accepted industry and regulatory standards to ensure compatibility with real-world financial systems.

The system uses RESTful API standards for communication between client and server, enabling efficient and stateless interaction over HTTP. Data exchange is primarily performed using JSON format, which ensures lightweight and structured communication.

In addition to technical standards, the system aligns with global AML compliance frameworks such as FATF guidelines and sanctions lists issued by OFAC, UN, and EU. This ensures that the system operates in accordance with regulatory expectations.

Standard data representations such as ISO country codes and consistent encoding formats are also used to maintain uniformity across datasets. These standards improve interoperability, reliability, and scalability of the system.

5.7 Mapping with IoTWF Reference Model Layers

Although the AML system is not a traditional IoT-based system, it can be conceptually mapped to the IoT World Forum (IoTWF) reference model to better understand its layered architecture.

The **data acquisition layer** corresponds to external sanctions sources and internal data inputs. The **data processing layer** includes ingestion, normalization, and indexing of data. The **application layer** consists of the screening APIs and business logic. Finally, the **business layer** represents risk categorization, monitoring, and alert generation.

This conceptual mapping helps in visualizing the system as a layered architecture, even though it is primarily a backend data processing system rather than a sensor-based IoT system.

5.8 Domain Model Specification

The domain model defines the key entities involved in the AML system and their relationships. These entities represent different components of the screening and monitoring process.

The primary entities include:

- **Sanctioned Entity:** Represents individuals or organizations present in sanctions lists, including attributes such as name, aliases, country, and date of birth
- **Customer Entity:** Represents users or entities being screened
- **Risk Category:** Defines classification levels such as high, medium, or low risk
- **Alert Entity:** Stores notifications generated when risk levels change
- **Source State:** Tracks the status of data ingestion and updates from external sources
- **API Usage Log:** Records request and response details for auditing

This structured domain model ensures consistency in data handling and improves clarity in system design.

5.9 Communication Model

The system follows a client-server communication model using RESTful APIs. All interactions between external systems and the AML platform are performed through HTTP-based requests and responses.

Clients send screening requests in JSON format, which are processed by the backend API. The system then interacts with Elasticsearch for search operations and PostgreSQL for data storage. The final results are returned to the client in a structured JSON response.

The communication model is stateless, meaning each request is processed independently. This improves scalability and allows the system to handle multiple concurrent requests efficiently.

5.10 IoT Deployment Level

Although the system is not an IoT application, its deployment model can be interpreted as a centralized backend deployment.

The AML system is deployed on a server environment where all processing, storage, and API services are hosted. External clients interact with the system through APIs, without requiring direct access to internal components.

This centralized deployment approach ensures better control, security, and performance. It also allows the system to scale easily by upgrading server resources or distributing components across multiple services if required.

5.11 Functional View

The functional view describes the key operations performed by the system and how they interact to produce desired outputs.

The main functions include:

- Data ingestion from multiple sources
- Data normalization and transformation
- Indexing and search operations
- Screening and scoring
- Risk categorization
- Ongoing monitoring and alert generation

Each function is handled by a dedicated module, ensuring efficient and organized processing. This structured approach improves system performance and maintainability.

5.12 Mapping IoT Deployment Level with Functional View

The deployment model and functional view are interconnected, as all system functions are executed within the centralized backend environment.

Data processing and indexing functions operate at the backend level, while API endpoints expose these functionalities to external users. Monitoring and alert generation functions run as background processes, ensuring continuous operation.

This mapping demonstrates how system functionality is effectively supported by the chosen deployment architecture.

5.13 Operational View

The operational view represents the runtime behaviour of the system and how it functions during execution.

When a request is received, it is processed through multiple stages including validation, candidate retrieval, scoring, and response generation. At the same time, background processes handle data updates and ongoing monitoring.

Logging mechanisms record system activity, including API usage and errors, which helps in monitoring performance and debugging issues. This ensures reliable and stable operation of the system in real-world scenarios.

5.14 Other Design Considerations

Additional design considerations include scalability, modularity, security, and performance optimization.

The use of a pipeline-driven architecture ensures efficient data processing, while the modular design allows easy extension of system components. Security mechanisms such as validation and controlled access help protect sensitive data.

The hybrid screening approach improves accuracy by combining search-based retrieval with contextual scoring. Additionally, automated data updates ensure that the system always operates on the latest sanctions information.

These design considerations collectively enhance the overall effectiveness and reliability of the AML sanctions screening system.

Chapter 6
HARDWARE, SOFTWARE AND SIMULATION

During the internship, I implemented and tested the Anti-Money Laundering (AML) Sanctions Screening System using standard hardware and open-source software tools. This chapter describes the hardware environment, development tools, backend implementation, and simulation approach used to validate the system.

6.1 Hardware

During the internship, the development and testing of the AML system were carried out on a personal laptop with standard configuration, which was sufficient for backend API development, data processing, and testing workflows.

The system configuration included an Intel i9 processor, 16 GB RAM, and SSD storage. This setup supported tasks such as API development, data ingestion, normalization, and screening operations efficiently.

For deployment, the system was hosted on a Linux-based server environment, which provided better stability and continuous runtime support. The server environment was configured to run backend services such as Flask APIs, PostgreSQL, and Elasticsearch.

For enterprise-level deployment, the system can be scaled using more powerful configurations such as multi-core processors, higher memory capacity (32-64 GB RAM), and high-speed storage. These enhancements enable efficient handling of large datasets, faster indexing, and improved performance for real-time screening.

6.2 Software Development Tools

During the internship, I used a combination of programming languages, frameworks, and tools to develop the AML backend system.

The primary programming language used was Python, due to its flexibility and strong support for backend development. The system was built using the Flask framework, which enabled the creation of RESTful APIs and efficient request handling.

Various libraries and tools were used to support different functionalities:

- Flask-CORS for handling cross-origin requests
- Requests library for fetching external sanctions data
- Pandas and OpenPyXL for processing structured data files
- Elasticsearch client for search and indexing operations
- SQLAlchemy for database interaction

For development and testing, tools such as Visual Studio Code and Postman were used. For deployment, Gunicorn was used to run the application in a production environment, ensuring stability and scalability.

A summary of major software tools used in this project can be seen in Table 6.1.

Table 6.1: Software Tools Used

Tool	Purpose
Python	Backend development
Flask	API framework
Elasticsearch	Search and indexing
PostgreSQL	Data storage
SQLAlchemy	Database interaction
Postman	API testing
VS Code	Code development

6.3 Software Code

During the internship, I developed the AML backend system using a modular code structure to ensure scalability and maintainability.

The system was organized into multiple components such as data ingestion, normalization, indexing, screening logic, and API handling. Each module was designed to perform a specific function, ensuring clear separation of concerns.

The backend exposes RESTful APIs for processing requests. These APIs allow operations such as screening, retrieving watchlists, configuring risk categories, and managing ongoing monitoring.

The system processes data through a pipeline:

- Source monitoring and data download
- Data normalization into unified format
- Indexing into Elasticsearch
- Screening using search and scoring
- Response generation

The modular design ensures that new features or data sources can be added easily without affecting existing components.

6.4 Simulation

Since direct access to real banking systems was not available during the internship, simulation-based testing was performed to validate the AML system. The simulation involved sending API requests using tools such as Postman with different input scenarios. These scenarios tested the system's ability to perform screening, retrieve results, and handle different conditions.

Table 6.2: Simulation Scenarios

Scenario	Input	Expected Output
Name Screening	Customer name	Matching sanctions records
Exact Match	Known entity	High-risk match result
Partial Match	Similar name	Scored match output
No Match	Random name	No risk detected
Monitoring	Saved customer	Alert on risk change

The results confirmed that the system successfully performed screening operations, generated accurate match scores, and handled different scenarios effectively. The simulation also validated the reliability of the system in handling real-world AML use cases.

The hardware and software setup used during the internship was sufficient to develop and test the AML backend system. The simulation approach helped in validating system functionality and ensured that the system performs reliably under different conditions.

Chapter 7
EVALUATION AND RESULTS

During the internship, I evaluated the performance and correctness of the Anti-Money Laundering (AML) Sanctions Screening System by conducting multiple test scenarios. The evaluation focused on verifying screening accuracy, request validation, system performance, and overall reliability of the backend pipeline across different real-world conditions.

7.1 Test Points

The evaluation phase for the AML system involved a multi-dimensional assessment to validate its accuracy, efficiency, and compliance with financial regulations. Test points were established across three primary domains to ensure comprehensive system evaluation

7.1.1 Accuracy Test Points

Accuracy testing focused on the effectiveness of the hybrid screening mechanism. This included evaluating the system's ability to correctly identify exact matches as well as approximate matches using name similarity techniques. Special attention was given to testing variations such as spelling differences, aliases, and partial matches.

The role of contextual attributes such as date of birth and country was also evaluated to ensure that false positives were minimized and only relevant matches were returned.

7.1.2 Performance Test Points

Performance evaluation focused on system latency and processing efficiency. Key metrics included API response time for real-time screening requests and the time required to process large datasets during indexing.

The system was tested to ensure that screening responses are generated quickly while maintaining accuracy. The use of Elasticsearch was particularly evaluated for its ability to handle large-scale search operations efficiently.

7.1.3 Compliance Test Points

Compliance testing ensured that the system adheres to AML regulatory requirements. This included verification of audit logs, data traceability, and proper handling of screening results.

The system was also tested for ongoing monitoring functionality, ensuring that scheduled re-screening of customer entities works correctly and generates alerts when risk levels change.

7.2 Test Plan

During the internship, I designed a structured test plan to validate the AML system under different scenarios. The testing process involved sending API requests using Postman with different input names and conditions. Each request was processed through the screening pipeline, and the output was verified against expected results. The various test cases can be observed in Table 7.1.

Table 7.1: Test Cases

Test Case	Input	Operation	Expected Result
TC1	Exact Name	Screening	High confidence match
TC2	Partial Name	Screening	Scored match result
TC3	Alias Name	Screening	Matching entity detected
TC4	Random Name	Screening	No match result
TC5	Known Entity	Screening	Correct risk classification
TC6	Invalid Input	Validation	Error response

The test plan ensured that all core functionalities of the AML system, including screening, validation, and monitoring, were thoroughly evaluated.

7.3 Test Results

The results obtained from testing confirm that the AML system performs efficiently across multiple scenarios. The system successfully handled both exact and approximate matching cases. The hybrid scoring mechanism improved accuracy by combining search-based retrieval with contextual validation.

Validation mechanisms correctly identified invalid inputs and returned appropriate error messages. The system also demonstrated consistent performance in handling multiple requests. A summary of the test results can be found in Table 7.2.

Table 7.2: Result Summary

Feature	Result
Screening Accuracy	Successful
Request Validation	Successful
Matching Logic	Effective
Error Handling	Successful
Performance	Efficient

The results demonstrate that the system is reliable and capable of handling AML screening operations effectively. The use of a pipeline-driven architecture and hybrid matching approach ensures accurate and efficient processing.

7.4 Insights

During the evaluation process, I gained several important insights into backend system design and AML compliance systems.

The use of a unified data normalization approach significantly reduced complexity and improved consistency across multiple data sources. The hybrid screening mechanism proved to be effective in balancing accuracy and performance by combining search capabilities with contextual scoring.

The modular architecture improved maintainability and allowed independent development of system components. Additionally, the use of Elasticsearch enhanced performance by enabling fast and efficient search operations.

The evaluation confirms that the AML system developed during the internship meets its objectives and performs reliably across different test scenarios. The system successfully demonstrates how automated backend architectures can be applied to real-world financial compliance problems.

Chapter 8
SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

During the internship, while developing the Anti-Money Laundering (AML) Sanctions Screening System, I considered not only the technical implementation but also the broader implications of such systems in financial and regulatory environments. Systems dealing with financial data must be designed responsibly, ensuring that they contribute positively to society while complying with legal and ethical standards. This chapter discusses the social, legal, ethical, sustainability, and safety aspects associated with the developed system.

8.1 Social Aspects

The AML sanctions screening system plays a significant role in strengthening financial security and transparency. By automating the process of identifying sanctioned individuals and entities, the system helps financial institutions prevent illegal activities such as money laundering, fraud, and terrorist financing.

The system contributes to improving operational efficiency by reducing the reliance on manual screening processes. Faster and more accurate screening enables institutions to provide better services to customers while maintaining regulatory compliance. This indirectly enhances user trust and confidence in financial systems.

Additionally, the system supports digital transformation in financial institutions by enabling seamless integration of multiple data sources and improving interoperability between systems. By handling heterogeneous data formats and providing a unified screening mechanism, the system simplifies complex backend operations and contributes to a more efficient financial ecosystem.

However, it is also important to consider the potential impact of false positives, where legitimate individuals may be flagged incorrectly. The system addresses this by using a hybrid

scoring mechanism that balances accuracy and sensitivity, allowing organizations to fine-tune thresholds according to their operational needs.

8.2 Legal Aspects

AML systems operate within a strict legal framework, as financial institutions are required to comply with international regulations and sanctions. The developed system aligns with global standards such as the Financial Action Task Force (FATF) guidelines and integrates sanctions data from authorities such as OFAC, United Nations (UN), and European Union (EU).

One of the key legal requirements in AML systems is maintaining auditability and traceability. The system ensures this by logging all API requests, responses, and processing activities. These logs provide a transparent record of system operations, which is essential for regulatory audits and compliance verification.

The system also incorporates data validation and structured processing to ensure that only correct and authorized data is handled. Secure communication through APIs and controlled access mechanisms helps protect sensitive financial data.

In real-world deployments, additional compliance requirements such as data protection laws (e.g., GDPR) and secure data storage practices would also need to be implemented. The current system provides a strong foundation for meeting such legal requirements.

8.3 Ethical Aspects

Ethical considerations are critical in systems that influence financial decisions and risk classification. The AML screening system is designed to ensure fairness, transparency, and accountability in its operation.

The use of a transparent scoring mechanism allows the reasoning behind each screening result to be understood. Unlike black-box models, the hybrid scoring approach clearly defines how name similarity and contextual attributes contribute to the final score. This improves trust and enables better decision-making by compliance officers.

The system also minimizes bias by using configurable thresholds and weighted scoring. This allows organizations to adjust parameters based on real-world requirements, reducing the risk of unfair treatment of individuals.

Furthermore, the system ensures that data is processed accurately without manipulation or loss of integrity. Proper validation mechanisms prevent incorrect or misleading data from being processed, ensuring that decisions are based on reliable information.

8.4 Sustainability Aspects

Sustainability in software systems involves efficient use of resources, long-term maintainability, and cost-effectiveness. The AML system is designed with these principles in mind.

The pipeline-driven architecture reduces redundant processing by ensuring that only updated data is ingested and indexed. This improves computational efficiency and reduces unnecessary resource usage.

The use of open-source technologies such as Python, Flask, PostgreSQL, and Elasticsearch significantly reduces development and deployment costs. This makes the system economically sustainable and accessible for organizations of different scales.

The modular design of the system supports long-term maintainability, allowing individual components to be updated or replaced without affecting the entire system. This reduces maintenance effort and ensures adaptability to future requirements.

Additionally, efficient backend processing contributes to reduced energy consumption, especially in large-scale deployments where resource optimization is critical.

8.5 Safety Aspects

Safety in AML systems involves ensuring reliability, data integrity, and secure operation. Since the system handles sensitive financial data, it is essential to prevent errors, unauthorized access, and system failures.

The system incorporates validation mechanisms to detect incorrect inputs at an early stage, preventing invalid data from propagating through the pipeline. Error handling mechanisms ensure that failures are managed gracefully without affecting overall system stability.

The modular architecture enhances safety by isolating system components. This ensures that failures in one module do not impact other parts of the system, improving overall reliability.

Logging mechanisms play a crucial role in monitoring system activity and identifying potential issues. Continuous operation is supported through backend services, ensuring that the system remains available and functional under different conditions.

For production environments, additional security measures such as authentication mechanisms and encrypted communication can further enhance system safety.

Chapter 9
CONCLUSION

During the internship, I successfully designed and implemented the Anti-Money Laundering (AML) Sanctions Screening System, a scalable and efficient backend solution developed to address critical challenges in financial compliance. The project focused on automating sanctions screening processes, which are essential for preventing financial crimes such as money laundering and fraud in modern financial systems.

The system was developed using a structured, pipeline-driven architecture that integrates multiple global sanctions data sources. These data sources, available in heterogeneous formats such as XML, JSON, and CSV, are processed through stages of ingestion, normalization, and indexing. By converting all data into a unified format and leveraging Elasticsearch for high-performance search operations, the system ensures efficient handling of large datasets and real-time screening capabilities.

A key contribution of the project is the implementation of a hybrid screening mechanism that combines search-based candidate retrieval with a weighted scoring algorithm. This approach improves accuracy by considering both name similarity and contextual attributes such as date of birth and country. As a result, the system effectively reduces false positives while maintaining high detection accuracy.

Through extensive testing and simulation, the system demonstrated reliable performance across multiple scenarios. It successfully handled exact and approximate matching cases, validated input data, and maintained consistent response times. The modular design ensures scalability and allows future enhancements without major structural changes.

The internship provided valuable practical experience in backend development, API design, and system architecture. It enhanced my understanding of real-world financial compliance systems and the challenges involved in handling large-scale, heterogeneous data. Additionally, it strengthened my ability to design efficient, scalable, and maintainable software solutions.

Overall, the developed AML sanctions screening system demonstrates how modern backend technologies and structured design approaches can be used to build effective and reliable compliance solutions. The project highlights the importance of automation, scalability, and accuracy in financial systems and provides a strong foundation for further development in this domain.

Future Scope

Although the system provides a solid foundation, several enhancements can be implemented to further improve its capabilities.

Future work may include integrating advanced authentication mechanisms such as JWT for improved security and access control. The system can also be extended to connect with real-world financial systems for live deployment and testing.

Enhancements in matching algorithms, including the use of machine learning techniques, can further improve accuracy and reduce false positives. Additionally, the development of a monitoring and analytics dashboard would provide better visibility into system performance and risk analysis.

Performance optimization and scaling strategies can be implemented to support large-scale deployments with higher data volumes and concurrent requests. These improvements would make the system more robust, secure, and suitable for enterprise-level applications.

REFERENCES

[1] Financial Action Task Force (FATF), International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation, FATF, Paris, France. Available: https://www.fatf-gafi.org

[2] OpenSanctions, OpenSanctions Data Platform Documentation. Available: https://www.opensanctions.org

[3] Elasticsearch B.V., Elasticsearch: The Definitive Guide, 2023. Available: https://www.elastic.co/guide

[4] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[5] D. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, Addison-Wesley, 1998. (Soundex Algorithm reference)

[6] PostgreSQL Global Development Group, PostgreSQL Documentation, Available: https://www.postgresql.org/docs/

[7] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002. (Data Pipeline & Architecture Concepts)

[8] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000. (REST APIs)

[9] T. White, Hadoop: The Definitive Guide, O'Reilly Media, 2015. (Distributed Processing & Scheduling Concepts)

[10] J. Leskovec, A. Rajaraman, and J. Ullman, Mining of Massive Datasets, Cambridge University Press, 2020. (Large-scale data processing and search concepts)

APPENDIX - A
PSEUDOCODE SNIPPETS

1. AML Screening Request Flow

Description: Accept a screening request, search Elasticsearch, enrich the summary with DB-driven risk details, and return paginated matches.

Pseudocode:

```
START screen request
READ JSON payload
IF payload is invalid THEN return 400
READ name from payload
IF name is empty THEN return 400
READ page and limit
VALIDATE page and limit are positive integers
CALL screener.screen with name, dob, countries, watchlists, entity_type, threshold,
exact_match, search_aliases
IF Elasticsearch transport fails THEN return 503
IF screening throws unexpected error THEN return 500
CALCULATE total_records from result list
CALCULATE total_pages and offset
SLICE results according to page and limit
SET pagination object in response
READ summary match_score
LOOK UP risk details from database using match_score
ADD risk_category, color, action_id, action_name into summary
CHECK screening_type from payload
IF screening_type is "name" THEN upsert ongoing name target
ELSE upsert ongoing customer target with name, dob, country, risk_category and score
IGNORE ongoing upsert failure but log it
RETURN final JSON response with query, summary, results and pagination
END screen request
```

2. External Source Refresh Pipeline

Description: Check upstream watchlists for changes, download updated files, and prepare them for normalization.

Pseudocode:

```
START source refresh cycle
FOR each external source such as OFAC, UN, EU, BIS, OpenSanctions
FETCH source metadata page, feed, or headers
EXTRACT latest available date or version string
NORMALIZE date into consistent format if needed
READ current source_state row for this source
IF no row exists THEN insert one
COMPARE latest available value with stored downloaded or known version
IF value changed THEN set has_update = TRUE
ELSE keep has_update as FALSE
END FOR
FOR each external source where has_update = TRUE
CHECK download window rules if applicable
DOWNLOAD source file into raw sources folder
SAVE or overwrite source file locally
UPDATE source_state latest_available and downloaded_version
SET last_downloaded_at to current UTC time
CLEAR last_error if download succeeded
IF download fails THEN record error in source_state
END FOR
HAND OFF downloaded sources to normalization stage
END source refresh cycle
```

3. Internal Source Config and Registration Flow

Description: Manage category-level config in DB and mark internal categories for pipeline processing when files or config change.

Pseudocode:

```
START internal source config flow
CREATE source category with source_name, source_description and file_format
SAVE category row into internal_source_categories
WHEN frontend requests get_source_config
LOOK UP category by source_name
IF category exists THEN return stored config fields and file_names
IF fields are empty THEN return null or defaults for frontend preload
WHEN frontend calls set_source_config
VALIDATE source_name exists
VALIDATE file_format and format-specific fields
SAVE sheet_name, has_headers, skip_rows, text_format, delimiter, name_fields,
dob_fields, dob_format, country_fields, records_path, record_path
UPDATE updated_at timestamp
MARK matching source_state row as source_type = internal and has_update = TRUE
WHEN frontend calls register_file
VALIDATE file_name and file_path
RESOLVE actual file path and ensure file exists
INFER uploaded file format from extension
VERIFY inferred format matches category format
APPEND or update file_name and file_path arrays in category row
MARK matching source_state row as has_update = TRUE
RETURN success message only
END internal source config flow
```

4. Internal Source Normalization and Indexing Flow

Description: Normalize all pending internal categories into JSONL and index all normalized datasets into Elasticsearch.

Pseudocode:

```
START internal normalization job
QUERY source_state for rows where source_type = internal and has_update = TRUE
FOR each source_name category returned
LOAD category config from internal_source_categories
LOAD all registered file_paths for that category
FOR each file in category
INFER file_format from config or extension
PARSE rows based on format rules: Excel, fixed text, delimited text, JSON, or XML
BUILD normalized entity records with uid, source, source_id, name, dob, country and
other fields
WRITE all normalized records into normalized/source_name.jsonl
COMPUTE fingerprint for the category output
UPDATE source_state downloaded_version, normalized_version, fingerprint,
last_normalized_at
SET has_update = FALSE when normalization succeeds
IF normalization fails THEN save last_error and keep category visible for retry
END FOR
START indexing job
CREATE a fresh Elasticsearch index with timestamped name
QUERY source_state rows that have normalized_version or fingerprint
FOR each source row, locate normalized/source_name.jsonl
IF file exists THEN bulk index its records into the new index
SET indexed_version from normalized_version for external or fingerprint for internal
UPDATE last_indexed_at in source_state
AFTER all sources complete, switch alias to the new index
END indexing job
```

APPENDIX - B

PROJECT SCREENSHOTS

Figure B.1 shows the AML Individual Name Screening page used to input and screen personal details against watchlists while Figure B.2 shows the AML Business Name Screening page used to screen company information against compliance databases.

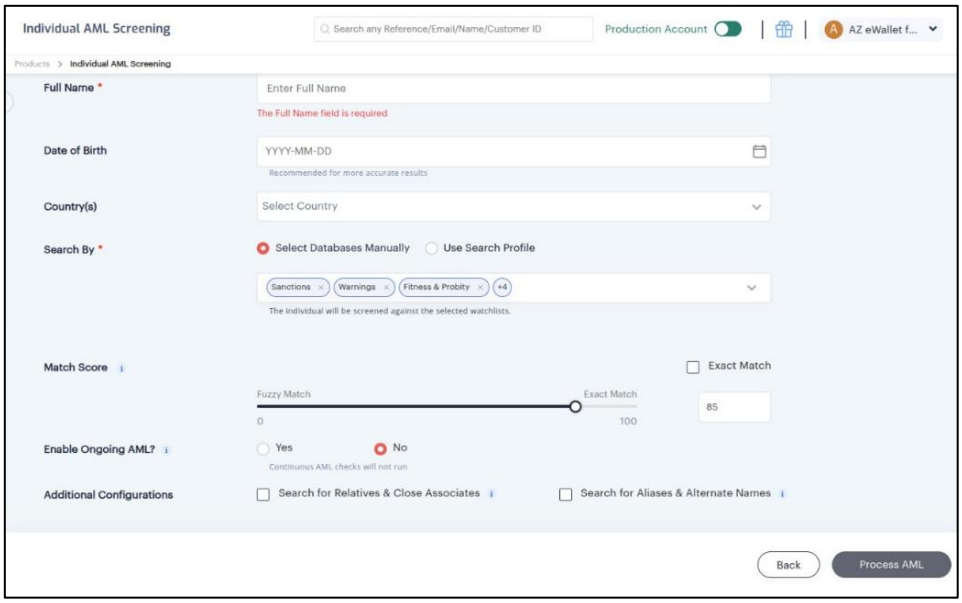

Figure B.1: AML Individual Name Screening Page

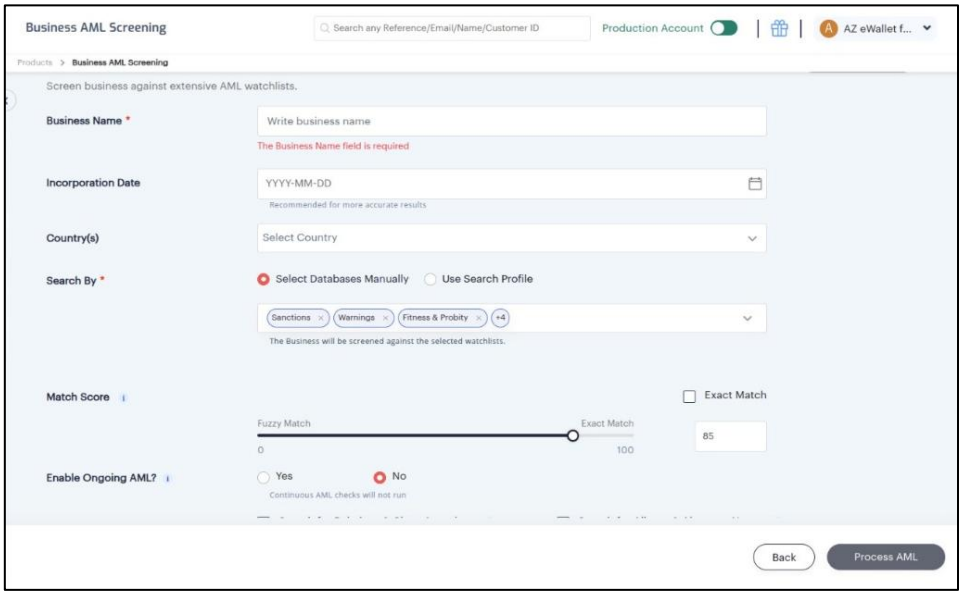

Figure B.2: AML Business Name Screening Page

Figure B.3 shows the AML Name Screening Results page displaying potential matches and their associated risk indicators.

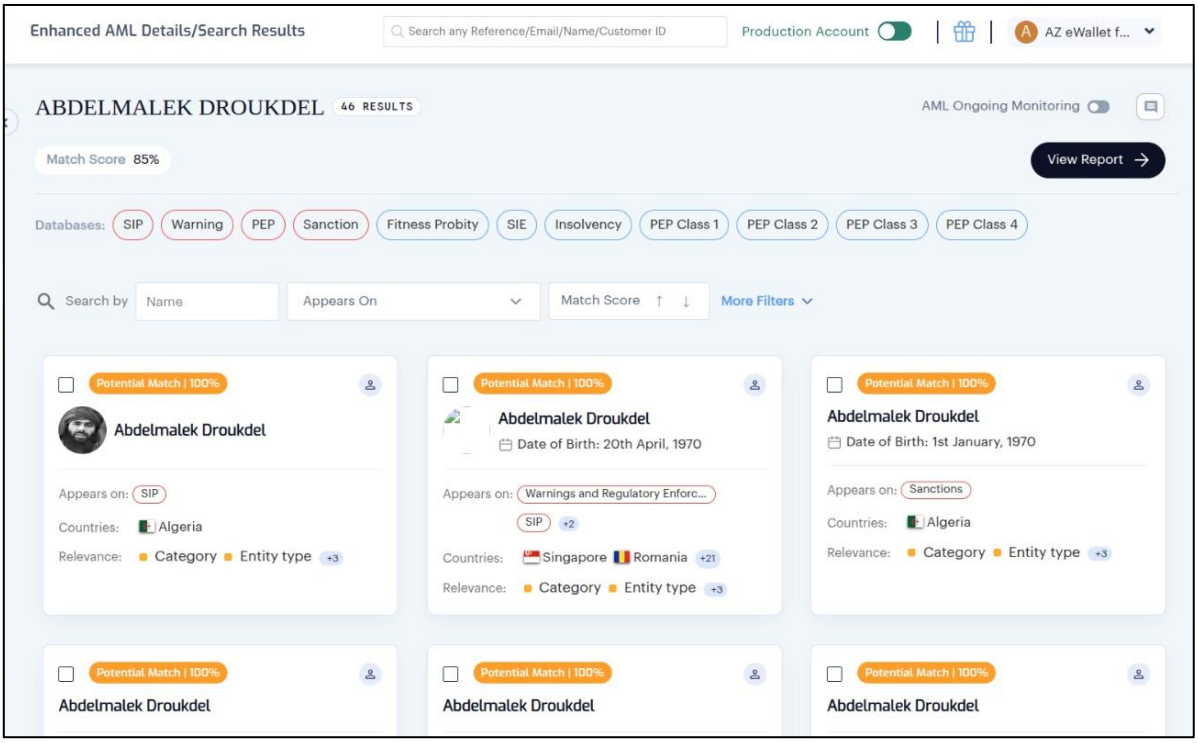

Figure B.3: AML Name Screening Results Page

Figure B.4 shows the AML Risk Categories range and action setup for defining risk thresholds and corresponding actions.

Figure B.4: AML Risk Categories Range and Action Setup